

Scenario 3: FOI Deadline Tracker

Production Hackathon — 2 days · teams of 2–4 · presentations at the end of each day

SCENARIO

The Department for Transport's central FOI team replaced their tracking spreadsheet with a small Flask app in March. Six caseworkers use it daily. It calculates the statutory 20-working-day response deadline, flags overdue requests, and has a search box the team asked for. It runs on a desktop machine under a desk. Backups are a USB stick, on Fridays, usually.

Then three things happened in one week. **Two more directorates asked for accounts** — that means multiple teams, real users, and personal data (requester names and addresses) properly handled. **The ICO confirmed an audit** of the department's FOI handling this autumn — audit trail, security posture, and deadline accuracy will all be examined. And **a request was answered late** even though the tracker showed it on time: it was received the Thursday before Easter, and the tracker doesn't know what a bank holiday is.

A missed statutory deadline is a breach of the Freedom of Information Act. The tracker's one job is the deadline. The deadline is wrong.

There is no AI in this system, and it doesn't need any. It needs to become a service the department can rely on — which makes this the purest operationalisation scenario of the three.

THE PROTOTYPE YOU'VE INHERITED

- A Flask CRUD app: log requests, view details, update status, search — with an SQLite database
- **The deadline calculation skips weekends but not bank holidays** — this has already caused one statutory breach
- The search box builds SQL by pasting user input directly into the query string — type carefully
- `secret_key = "dev"`, `debug=True`, no authentication, no audit log of who changed what
- The database lives next to the code, gets wiped by `seed.py`, and its backup strategy is Gary
- Runs with `python seed.py` then `python app.py`. It works — try it, including the search box.

YOUR GOAL

Make this production-grade. The autumn ICO audit is your definition of done: could this system, and the way you run it, pass an auditor asking about deadline accuracy, data protection, access control, and recoverability? Spend two days closing the gap that matters most.

By the end of Day 2 you should be able to stand up and say: here is what we found, here is what we prioritised and why, here is the evidence it works, and here is what we would do next.

DIRECTIONS YOU COULD TAKE

These are signposts, not a checklist. You will not do all of them — part of the exercise is deciding what matters most, in what order, and being able to defend that call.

- **Correctness first** — fix the deadline calculation using the GOV.UK bank holidays API (a public JSON feed), and build the test suite that proves it: Easter, Christmas, a request received on a bank holiday, the exact ICO day-counting rules. This is the single highest-value fix in the codebase.
- **Security** — parameterised queries for the injection, real secret management, authentication and per-user accounts (two directorates are coming), and an answer to 'who changed this record and when' — an audit log is exactly what the ICO will ask for.
- **Operations** — a deployment story that isn't Gary's desktop: containerise it where your machine allows, script the setup where it doesn't, automate backups and — crucially — prove a restore actually works.
- **Data protection** — requester names and addresses are personal data under UK GDPR. Retention policy, access control, and what happens when a requester asks for their own data to be deleted.
- **A CI pipeline** — tests on every change, a security scan, and a deploy step. Everything W7 covered, applied to a real service. This codebase has zero tests today; the deadline logic alone justifies the pipeline.
- **Features that operationalise** — email alerts as deadlines approach, an exceptions report for the FOI officer's Monday meeting, statistics for the department's annual FOI return, multi-team support with data separation.

PRESENTATIONS

- **End of Day 1 (5 minutes):** your priority call — what you found when you audited the prototype, what you decided matters most, and what you've done so far. Alternatively, a short explainer on a concept you researched to unblock yourselves (e.g. how prompt caching pricing actually works, or what the Batch API changes).
- **End of Day 2 (10 minutes):** what you shipped. Demonstrate the difference between the prototype you inherited and the system you're handing back — and name, honestly, what you would do next if you had a third day.

STARTING POINTS

- [GOV.UK bank holidays API \(JSON\)](#)
- [ICO guidance — time limits for FOI compliance](#)
- [OWASP SQL injection prevention cheat sheet](#)
- [GDS Way](#)

QUESTIONS WORTH ASKING YOURSELVES

1. A request arrives on Maundy Thursday. What deadline does the current code produce, what's the correct one, and what test proves your fix?
2. What exactly could someone do through the search box today? Demonstrate it safely, then close it.
3. The ICO auditor asks: 'who has accessed this requester's record?' What's your answer today, and what is it after your changes?

4. Gary's machine dies on a Wednesday. Walk through the recovery, step by step, with the current setup and with yours.

PRACTICAL NOTES

- **No Docker or Git on your machine?** Fine. Containerisation and version control are directions, not prerequisites — everything runs with plain python. If you want version control without Git, keep dated copies or use a cloud editor; if you want to explore containers, pair with a team member whose machine has Docker.
- **This scenario needs no API key at all** — which also makes it the most portable choice if your machine is heavily locked down. The bank holidays API is a plain public JSON feed.